

Name : Sunny

CU240251760

Sec: B

Roll No : 58

Computer Vision

Labsheet – 8

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture("video1.mp4")

if not cap.isOpened():
    print("Error: Video not found")
else:
    print("Video loaded successfully")

Video loaded successfully

ret, old_frame = cap.read()

if not ret:
    print("Unable to read video")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 5))
plt.imshow(cv2.cvtColor(old_frame, cv2.COLOR_BGR2RGB)) plt.title("First
Frame")
plt.axis("off")
plt.show()
```

First Frame



```
feature_params = dict(  
    maxCorners=100,  
    qualityLevel=0.3,  
    minDistance=7,  
    blockSize=7  
)  
  
p0 = cv2.goodFeaturesToTrack(  
    old_gray,  
    mask=None,  
    **feature_params  
)
```

```
print("Feature points detected:", len(p0))
```

Feature points detected: 100

```
lk_params = dict(  
    winSize=(15, 15),  
    maxLevel=2,  
    criteria=(  
        cv2.TERM_CRITERIA_EPS |  
        cv2.TERM_CRITERIA_COUNT,  
        10,  
        0.03  
    ))
```

```

mask = np.zeros_like(old_frame)

ret, frame = cap.read()

frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

p1, status, error = cv2.calcOpticalFlowPyrLK( old_gray,
frame_gray,
p0,
None,
**lk_params
)

good_new = p1[status == 1]
good_old = p0[status == 1]

print("Tracked points:", len(good_new))

Tracked points: 100

result = frame.copy()

for new, old in zip(good_new, good_old): x1, y1 =
new.ravel()
x2, y2 = old.ravel()

cv2.line(
mask,
(int(x2), int(y2)),
(int(x1), int(y1)),
(0, 255, 0),
2
)

cv2.circle(
result,
(int(x1), int(y1)),
4,
(0, 0, 255),
-1
)

output = cv2.add(result, mask)

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Object Tracking") plt.axis("off")

```

plt.show()



```
displacements = []
```

```
for new, old in zip(good_new, good_old):
```

```
    dx = new[0] - old[0]
```

```
    dy = new[1] - old[1]
```

```
    distance = np.sqrt(dx**2 + dy**2)
```

```
    displacements.append(distance)
```

```
if len(displacements) > 0:
```

```
    average_displacement = np.mean(displacements)
```

```
    print("Average Displacement:", round(float(average_displacement), 2))
```

```
else:
```

```
    print("No points tracked")
```

```
Average Displacement: 0.61
```

```
flow = cv2.calcOpticalFlowFarneback(
```

```
    old_gray,
```

```
    frame_gray,
```

```
    None,
```

```
    0.5,
```

```
    3,
```

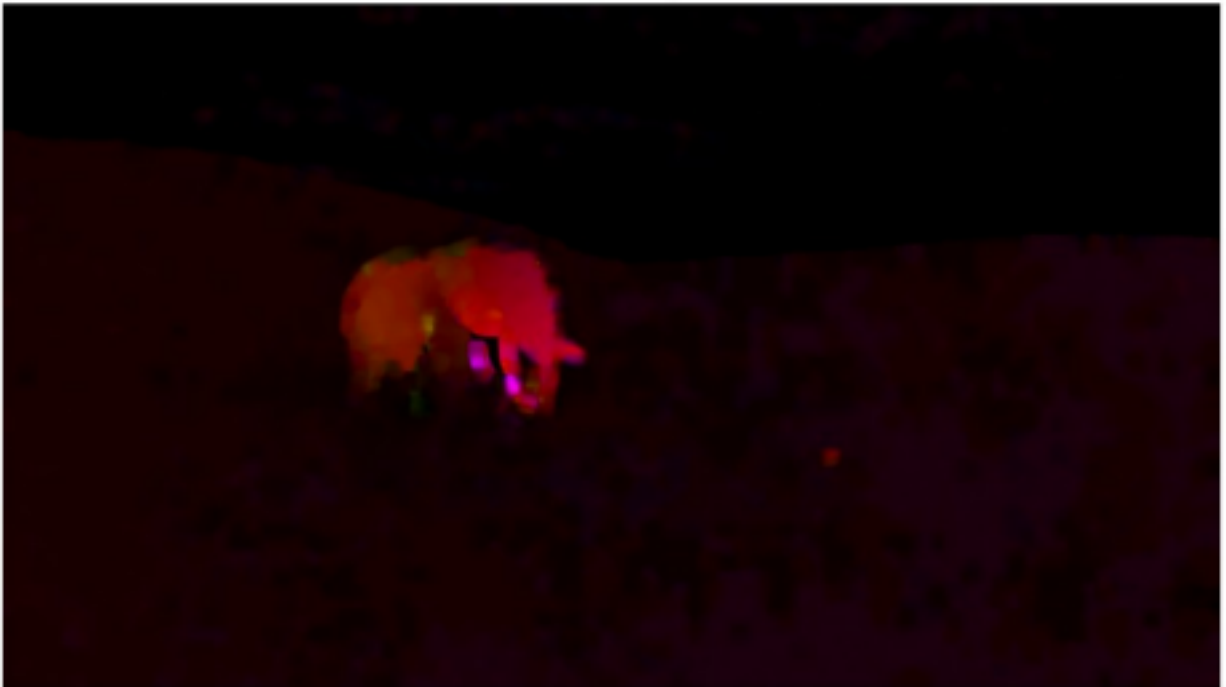
```
    15,
```

```
    3,
```

```
    5,
```

```
1.2,  
0  
)  
  
magnitude, angle = cv2.cartToPolar(  
    flow[..., 0],  
    flow[..., 1]  
)  
  
hsv = np.zeros_like(frame)  
hsv[..., 1] = 255  
  
hsv[..., 0] = angle * 180 / np.pi / 2  
hsv[..., 2] = cv2.normalize(  
    magnitude,  
    None,  
    0,  
    255,  
    cv2.NORM_MINMAX  
)  
  
dense_flow = cv2.cvtColor(  
    hsv,  
    cv2.COLOR_HSV2RGB  
)  
  
plt.figure(figsize=(10, 6))  
plt.imshow(dense_flow)  
plt.title("Farneback Dense Optical Flow")  
plt.axis("off")  
plt.show()
```

Farneback Dense Optical Flow



```
print("Optical Flow Tracking Comparison")
print("-----")
print("Lucas-Kanade : Sparse feature tracking")
print("Farneback : Dense pixel-wise tracking")
print("Lucas-Kanade : Faster and suitable for selected objects") print("Farneback :
Provides detailed motion information")
```

Optical Flow Tracking Comparison

Lucas-Kanade : Sparse feature tracking

Farneback : Dense pixel-wise tracking

Lucas-Kanade : Faster and suitable for selected objects Farneback :

Provides detailed motion information

cap.release()

print("Experiment completed successfully.")

Experiment completed successfully.

Theory Answers

Q1. How does object tracking differ from object detection?

Object detection identifies and locates objects in a single image or video frame. Object tracking follows the same object across multiple consecutive frames and determines its movement and trajectory. Detection tells us *what and where the object is*, while tracking tells us *where the object moves over time*.

Q2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the movement of pixels between consecutive video frames. Selected feature points are detected using methods such as Shi-Tomasi Corner Detection and tracked using Lucas-Kanade Optical Flow. The movement of these points is used to calculate the object's direction, displacement, and trajectory in real time.

Q3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi Corner Detection is used to identify strong and distinctive feature points in an image. These points are suitable for tracking because their movement can be detected easily between frames. Lucas-Kanade then tracks these selected points to estimate the object's motion.

Q4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow is suitable because it provides information about the direction and magnitude of motion between consecutive frames. It can continuously track movement and help calculate displacement, trajectory, and speed. Therefore, it is useful for analyzing dynamic scenes and moving objects.

Q5. What challenges arise while tracking fast-moving or partially occluded objects?

Fast-moving objects can move a large distance between consecutive frames, making feature tracking difficult. Partial occlusion can hide important feature points from the camera. Changes in illumination, motion blur, and camera movement can also reduce tracking accuracy.

Q6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow tracks motion by analyzing changes between consecutive frames and generally requires less computational power. Deep learning-based trackers can recognize and track objects using trained models and are usually more robust to complex scenes. However, deep learning methods generally require more computational resources and suitable training data.

Q7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Optical Flow can estimate the movement of vehicles, pedestrians, and other objects in traffic scenes. It can help determine their direction and speed and detect changes in their movement. In autonomous driving, this information can support navigation, obstacle detection, and understanding of the surrounding environment.

Q8. Explain the effect of camera motion on Optical Flow estimation.

Camera movement causes many or all pixels in the scene to appear to move, even when objects themselves are stationary. This can make it difficult to distinguish object motion from camera motion. As a result, the estimated Optical Flow may contain unwanted motion vectors and reduce tracking accuracy.

Q9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

Five applications are:

1. Video surveillance
2. Traffic monitoring
3. Autonomous navigation
4. Human activity recognition
5. Sports analytics

Optical Flow is also useful in robotics and object tracking.

Q10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

In surveillance, Optical Flow helps detect and track moving objects. In robotics, it provides information about environmental movement and helps with navigation. In human activity recognition, motion patterns can be analyzed to identify activities such as walking, running, or other movements. Thus, Optical Flow helps systems understand dynamic scenes.